МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
Кафедра інформаційних технологій

ЛАБОРАТОРНА РОБОТА №3

З дисципліни: «Об’єктно-орієнтоване та подійно-орієнтоване програмування»
На тему: «Особливості використання змінних і типів даних у Java»

Роботу виконав:
студент II курсу групи АД-251
спеціальності F6
Горбунова А.А.

Роботу перевірив:
к т.н., доцент Рудніченко М.Д.

Оцінка: _____

ДУОП, 2026

1 МЕТА РОБОТИ ТА ЗАВДАННЯ

Мета роботи: Метою лабораторної роботи є формування практичних навичок роботи зі змінними та типами даних у мові програмування Java. У процесі виконання роботи студент повинен засвоїти особливості оголошення та ініціалізації змінних, навчитися розрізняти примітивні та об’єктні типи даних, використовувати числові, символьні та логічні значення, виконувати перетворення типів, працювати з рядками та масивами, а також застосовувати отримані знання під час створення простих класів і об’єктів.

Особлива увага приділяється розумінню взаємозв’язку між типом змінної, значенням, яке вона зберігає, та операціями, які дозволено виконувати над цим значенням. Наприкінці лабораторної роботи отримані знання застосовуються під час розроблення невеликої об’єктно-орієнтованої програми.

Завдання на лабораторну роботу:

1) Ознайомитися з матеріалами лабораторної роботи.

2) Перевірити роботу всіх представлених прикладів, результати привести в вигляді скріншотів і тексту.

3) **Додаткове індивідуальне завдання з файлу лаб3.docx. Варіант №7.** Розробити програму для обліку фільмів. Створити клас Movie, який містить назву, жанр, рік випуску, тривалість у хвилинах та рейтинг. Створити масив фільмів. Вивести всі фільми та визначити фільм із найвищим рейтингом.

4) Оформити звіт і відповісти на контрольні питання.

2 ПЕРЕВІРКА БАЗОВИХ ТЕОРЕТИЧНИХ ВІДОМОСТЕЙ ТА ДЕМОНСТРАЦІЙНИХ ПРИКЛАДІВ

2.1 Теоретичний аналіз особливостей типів даних у мові Java

Згідно з програмою досліджень, у межах лабораторної роботи було проведено детальний аналіз взаємозв'язку між типами змінних, примітивними та об'єктними структурами даних у мові Java. Було досліджено такі фундаментальні аспекти:

– **Особливості цілочисельного ділення типів `int`:** При діленні двох об'єктів типу `int` результат завжди округляється до цілого числа у бік нуля, а дробова частина повністю відкидається. Для отримання точного результату з плаваючою комою хоча б один з операндів повинен явно мати тип `double` або `float`.

– **Похибка представлення чисел з плаваючою комою (`double`):** Через специфіку двійкового представлення дробових чисел у пам'яті комп'ютера за стандартом IEEE 754, деякі арифметичні операції можуть виконуватися з незначною похибкою (наприклад, операція $0.1 + 0.2$ повертає значення 0.30000000000000004). Цей фактор є критичним при проектуванні точних обчислювальних систем.

– **Ізоляція та порівняння об'єктного типу `String`:** На відміну від примітивних типів даних, змінні класу `String` зберігають не саме значення, а посилання на область пам'яті об'єкта у кучі (*Heap*). Через це застосування оператора == порівнює виключно адреси самих посилань, а не їхній зміст. Для посимвольного порівняння вмісту рядків обов'язковим є використання вбудованого методу `equals()`.

2.2 Перевірка виконання демонстраційних прикладів у терміналі

Для перевірки вищезазначених теоретичних тверджень та виконання обов'язкового пункту №2 завдання було запущено службовий метод `TheoryExamples.runAllExamples()`. На рисунку **??** представлено скриншот відповідної ділянки консольного виведення у терміналі розробки.

Рисунок 2.1 – Ділянка консолі з результатами перевірки базових прикладів із методички лаб3.docx

Як видно з результатів тестування (рис. **??**), всі системні логи компілятора та віртуальної машини Java повністю підтвердили особливості обробки пам’яті та примітивів, описані в теоретичній частині роботи.

2.3 Лістинг службового класу `TheoryExamples.java`

Нижче наведено повний вихідний код створеного компонента, що інкапсулює у собі демонстраційні алгоритми перевірки мовних конструкцій.

3 ОПИС ПОБУДОВИ КОДУ ІНДИВІДУАЛЬНОГО ЗАВДАННЯ. ВАРІАНТ №7

3.1 Об’єктно-орієнтована архітектура та декомпозиція компонентів

Згідно з індивідуальним Варіантом №7, було розроблено консольну програму для обліку фільмів у кінокаталозі. На відміну від процедурного стилю мови С, проєктування архітектури виконано із суворим дотриманням принципів успадкування (*Inheritance*), інкапсуляції та поліморфізму.

Кодову базу розподілено по окремих файлах простору імен `ad251.gorbunova` за такою логічною структурою:

1) **Базовий суперклас `Media.java`:** Виступає в ролі абстрактного фундаменту предметної області. Він інкапсулює у собі захищене поле `protected String title` (назва витвору) та базовий віртуальний метод `printInfo()` для представлення медіа-продукції у консолі.

2) **Дочірній клас `Movie.java`:** Розширює функціональність базового класу за допомогою ключового слова `extends`. Клас додає специфічні інкапсульовані характеристики кінострічки: приватні поля жанру (`String genre`), року випуску (`int year`), тривалості (`int duration`) та рейтингу (`double rating`).

3) **Сервісний аналізатор масивів `MovieCatalog.java`:** Компонент, який інкапсулює всередині себе масив посилань на об’єкти базового типу `Media[]`. Він повністю ізолює логіку обробки складних структур даних (пошук максимуму, ітераційні цикли) від користувача.

4) **Головний клас `Main.java`:** Точка входу в програму (`main()`), яка координує створення об’єктів та запускає демонстраційні тести.

3.2 Реалізація механізмів успадкування та поліморфізму в коді

Під час проектування взаємодії класів було детально опратовано два найважливіші інструменти мови Java:

– **Роль ключового слова `super`:** Конструктор дочірнього класу `Movie` першим рядком обов'язково викликає конструктор батьківського класу за допомогою інструкції `super(title)`. Це гарантує правильну послідовну ініціалізацію полів об'єкта в кучі (*Heap*) під час роботи оператора `new`.

– **Динамічний об'єктний поліморфізм:** Метод `printCatalog()` класу `MovieCatalog` виконує ітераційний перебір масиву `Media[]`. Під час виклику `item.printInfo()`, завдяки механізму пізнього (динамічного) зв'язування, віртуальна машина Java (JVM) на льоту визначає фактичний тип об'єкта в пам'яті й викликає переозначений метод `@Override` дочірнього класу `Movie`, а не базового суперкласу.

– **Безпечне приведення типів даних:** Для виконання індивідуального завдання з пошуку фільму з найвищим рейтингом, у класі `MovieCatalog` застосовано логічний оператор `instanceof`. Він перевіряє, чи є об'єкт у масиві базових посилань екземпляром класу `Movie`, після чого виконується безпечне явне приведення типів `(Movie) item` для доступу до приватного геттера `getRating()`.

4 ЛІСТИНГ ПРОГРАМНОГО КОДУ

У цьому розділі наведено повний вихідний код. Код оформлено згідно зі стандартами *Oracle Code Conventions* та містить вичерпну документацію у форматі *JavaDoc*.

4.1 Клас Main.java

```
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/* Імпорт стандартних колекцій Java Collections Framework*/
import java.util.ArrayList;    // Динамічний лінійний список студентів групи
import java.util.HashMap;      // Асоціативна хеш-мапа для індексного пошуку за кодом
import java.util.HashSet;      // Множина для збереження унікальних навчальних дисциплін
import java.util.List;         // Інтерфейс впорядкованих послідовностей даних
import java.util.Map;          // Інтерфейс для структур зберігання пар "ключ - значення"
import java.util.Set;          // Інтерфейс унікальних контейнерів без дублікатів

/**
 * Головний керуючий клас-оркестратор Лабораторної роботи №4.
 * Координує виконання спільної практичної частини та індивідуального Варіанту №7.
 *
 * @author Анастасія Горбунова
 * @version 4.0 15 Oct 2026
 */
public class Main {

    /**
     * Головна точка входу в додаток.
     *
     * @param args аргументи командного рядка CLI
     */
    public static void main(String[] args) {
        System.out.print("=== Лабораторна робота №4 (Облікові структури даних) ===\n\n");
        System.out.print("Тема: Робота з обліковими структурами даних\n\n");

        // ========================================================================
        // 1. ЗАГАЛЬНА ПРАКТИЧНА ЧАСТИНА (Робота зі списками Student через List)
        // ========================================================================
        System.out.print("--- 1. Тестування структури List<Student> ---\n");
        List<Student> students = new ArrayList<>();
        students.add(new Student("Олена Коваль", 20, 91.5));
        students.add(new Student("Андрій Петренко", 21, 87.2));
        students.add(new Student("Марія Шевченко", 19, 95.1));

        for (Student student : students) {
            student.printInfo();
        }

        double sum = 0;
        for (Student student : students) {
            sum += student.getAverageGrade();
        }
        System.out.printf("Середній бал навчальної групи: %.2f\n\n", (sum / students.size()));
    }
}
```

4.2 Клас Movie.java

4.3 Клас Media.java

4.4 Клас MovieCatalog.java

5 РЕЗУЛЬТАТИ РОБОТИ ПРОГРАМИ

Для повної верифікації розроблених алгоритмів, контролю стану об'єктів та перевірки коректності роботи з примітивними та об'єктними типами даних, а також демонстрації механізмів успадкування та поліморфізму було здійснено запуск програми в єдиному циклі збирання середовища VS Code. На рисунку **??** представлено фінальний вивід результатів у консоль Windows PowerShell.

Рисунок 5.1 – Вигляд терміналу після успішного виконання всіх завдань Лабораторної роботи №3

На основі отриманих даних терміналу (рис. **??**) проведено детальний покроковий аналіз результатів тестування:

1) **Результати перевірки теоретичних прикладів:**

– При діленні цілих чисел `10 / 3` тип `int` повністю відкинув дробову частину, вивівши результат `3`. Використання типу `double` у виразі `10.0 / 3` дозволило отримати точний дробовий результат `3.3333333333333335`.

– Операція додавання `0.1 + 0.2` наочно продемонструвала системну похибку двійкового представлення дробових чисел у пам'яті комп'ютера, вивівши результат `0.30000000000000004`.

– Інспекція методів об'єктного типу `String` підтвердила правильність обробки рядків: метод `.length()` зафіксував довжину `16` символів, а метод `.charAt(0)` безпомилково витягнув перший символ `'J'`.

– Порівняння двох різних об'єктів рядків через оператор == повернуло значення `false` (оскільки порівнювалися адреси посилань у пам'яті), тоді як логічний метод `.equals()` повернув істинне значення `true`, підтвердивши повний збіг вмісту об'єктів.

2) **Результати виконання індивідуального завдання:** Сервісний компонент `MovieCatalog` успішно проітерував масив об'єктних посилань базового типу `Media[`

задіявши механізм динамічного об’єктного поліморфізму. У консоль було виведено повний відформатований список із 5 кінофільмів із зазначенням їхніх специфічних інкапсульованих характеристик (жанр, рік, хронометраж, рейтинг).

3) **Пошук лідера рейтингу:** Розроблений лінійний алгоритм пошуку максимуму, що використовує безпечне приведення типів за допомогою оператора `instanceo` безпомилково визначив фільм із найвищим рейтингом. Ним стала кінострічка **«Темний лицар»** із максимальною оцінкою у **`9.0`** балів.

6 ВІДПОВІДІ НА КОНТРОЛЬНІ ЗАПИТАННЯ

1. Що таке змінна в мові Java та яку інформацію визначає її тип.

Змінна — це іменована область пам'яті, призначена для зберігання певного значення під час виконання програми. Тип змінної визначає множину значень, які вона може набувати, обсяг виділеної пам'яті, а також набір операцій, які дозволено виконувати над цим значенням.

2. Чим відрізняються примітивні типи від об'єктних типів.

Примітивні типи безпосередньо зберігають свої прості значення у стеку (*Stack*) і не є об'єктами. Об'єктні (посилальні) типи зберігають лише посилання (адресу) на об'єкт, який фізично розміщується в динамічній пам'яті — кучі (*Heap*). Крім того, об'єктні типи можуть містити методи та набувати значення `null`.

3. Які вісім типів даних у Java є примітивними. У мові Java існує рівно вісім типів даних у Java є примітивними:

- Цілочисельні: `byte`, `short`, `int`, `long`;
- Дробові (з плаваючою комою): `float`, `double`;
- Символьний: `char`;
- Логічний: `boolean`.

4. Для чого використовується тип `int`.

Тип `int` використовується для представлення 32-бітних цілих чисел зі знаком. Він є стандартом за замовчуванням у Java для більшості звичайних математичних обчислень, лічильників циклів та збереження дискретних кількостей об'єктів.

5. У чому полягає відмінність між `float` та `double`.

Тип `float` є типом одинарної точності, займає 32 біти в пам'яті й потребує суфікса `f` при оголошенні літералів. Тип `double` є типом подвійної точності,

займає 64 біти, забезпечує значно вищу точність обчислень і є стандартом за замовчуванням для дробових чисел у Java.

6. Для чого використовується тип `boolean`.

Логічний тип `boolean` призначений виключно для зберігання результатів умовних виразів та логічних операцій. Він може набувати лише одного з двох можливих значень: `true` (істина) або `false` (хибність).

7. Чим відрізняється `char` від `String`.

Тип `char` є примітивним, займає 16 біт і зберігає строго один символ Unicode в одинарних лапках (наприклад, `'A'`). Клас `String` є об'єктним типом даних, оперує послідовністю символів у подвійних лапках (наприклад, `"Java"`) та інкапсулює методи для роботи з текстом.

8. Чому рядок є об'єктним типом.

Рядок у Java є екземпляром повноцінного класу `String`, а не просто масивом примітивів чи базовим типом. Рядки мають власні методи обробки, підтримують механізм незмінності (*Immutability*) та зберігаються у кучі, де оптимізується спеціальним пулом рядків (*String Pool*).

9. Що таке класи-обгортки.

Класи-обгортки (*Wrapper Classes*) — це спеціальні класи з пакету `java.lang`, які представляють примітивні типи даних у вигляді повноцінних об'єктів. Кожному примітиву відповідає свій об'єктний аналог (наприклад, `int` → `Integer`, `double` → `Double`), що дозволяє використовувати їх там, де потрібні виключно об'єкти.

10. Що таке автоупаковування та автораозпаковування.

Автоупаковування (*Autoboxing*) — це автоматичне перетворення примітивного типу в об'єкт відповідного класу-обгортки (наприклад, присвоєння значення `int` змінній типу `Integer`).

Авторозпаковування (*Auto-unboxing*) — зворотний процес, при якому об'єкт

класу-обгортки автоматично трансформується у примітивне значення при виконанні арифметичних операцій.

11. Що відбувається під час перетворення `double` у `int`.

Під час явного приведення типу `double` у `int` відбувається примусове відсікання (відкидання) всієї дробової частини числа, а не його математичне округлення. Значення просто урізається до цілого числа, що може призвести до втрати точності даних (наприклад, `(int) 25.75` перетвориться на `25`).

12. Для чого використовується ключове слово `final`.

Ключове слово `final` використовується для створення констант, тобто значень, які не можуть бути змінені після їхньої першої ініціалізації. Оголошення змінної або поля класу з модифікатором `final` блокує будь-які спроби повторного переприсвоєння значення на етапі компіляції.

13. Чим відрізняється локальна змінна від поля класу.

Локальна змінна оголошується безпосередньо всередині методу або блоку коду, існує лише під час його виконання і зберігається у стеку. Вона не має значень за замовчуванням і вимагає обов'язкової ручної ініціалізації перед використанням. Поле класу оголошується всередині тіла класу, визначає стан об'єкта у кучі й автоматично ініціалізується значеннями за замовчуванням при створенні об'єкта.

14. Що таке конструктор класу.

Конструктор — це спеціальний метод класу, який автоматично викликається оператором `new` під час створення нового об'єкта для ініціалізації його початкового стану. Конструктор не має типу повертаємого значення, а його назва повинна строго збігатися з ім'ям самого класу.

15. Для чого використовується ключове слово `this`.

Ключове слово `this` є прихованим посиланням на поточний екземпляр об'єкта,

всередині якого викликається метод або конструктор. Найчастіше воно застосовується для розв'язання конфлікту імен, коли назви вхідних аргументів конструктора повністю збігаються з іменами полів класу (наприклад, `this.name = name`).

16. Що таке об'єкт.

Об'єкт — це конкретний будівельний блок об'єктно-орієнтованої програми, який створюється в динамічній пам'яті (кучі), є реальним втіленням (екземпляром) свого абстрактного класу та має власний унікальний стан і визначену поведінку.

17. Чим відрізняється примітивна змінна від змінної об'єктного типу.

Примітивна змінна безпосередньо містить у собі конкретне просте значення (число, символ або логічний прапорець). Змінна об'єктного типу містить у собі лише адресу (посилання), яка вказує на місце розробки реального об'єкта у кучі. При копіюванні примітиву копіюється саме значення, а при копіюванні об'єктної змінної — лише адреса посилання на той самий об'єкт.

18. Що означає значення `null`.

Значення `null` — це спеціальний літерал, який може набувати будь-яка змінна об'єктного (посилального) типу. Воно офіційно означає, що дана змінна наразі є порожньою і не посилається на жоден реальний об'єкт у кучі.

19. Чому звернення до методу через посилання зі значенням `null` може спричинити `NullPointerException`.

Оскільки значення `null` вказує на відсутність об'єкта, спроба викликати через таке порожнє посилання будь-який метод або звернутися до поля змушує віртуальну машину Java шукати дані за несучуючою адресою. Не маючи фізичного об'єкта для виконання операції, JVM негайно зупиняє роботу програми та генерує критичну помилку `NullPointerException`.

20. Що таке масив.

Масив у мові Java — це повноцінний об’єктний тип даних, який є впорядкованою структурою для зберігання фіксованої кількості елементів одного типу. Розмір масиву жорстко визначається під час його створення, не може змінюватися в процесі роботи програми, а доступ до його елементів здійснюється за допомогою індексів.

21. Чому індексація масивів у Java починається з нуля.

Індексація з нуля базується на низькорівневій логіці обчислення адрес пам’яті процесором. Індекс елемента масиву насправді є коефіцієнтом **зміщення (offset)** відносно початкової адреси першого елемента. Оскільки перший елемент знаходиться безпосередньо на початку виділеного блоку пам’яті, його зміщення дорівнює нулю.

22. Чим масив об’єктів відрізняється від масиву примітивних значень.

Масив примітивних значень безпосередньо містить усередині своїх комірок самі прості значення (наприклад, числа типу `int`). Масив об’єктів містить у своїх комірках не самі об’єкти, а лише **адреси посилань** на них. Самі об’єкти при цьому створюються та зберігаються окремо в кучі (*Heap*), а за замовчуванням комірки масиву об’єктів ініціалізуються значенням `null`.

23. Що таке інкапсуляція.

Інкапсуляція — це один із трьох фундаментальних принципів ООП, який полягає в логічному об’єднанні даних (полів стану) та методів для їхньої обробки всередині однієї сутності (класу), а також у прихованні внутрішньої реалізації об’єкта від прямого неконтрольованого доступу з боку зовнішніх компонентів програми.

24. Для чого поля класу оголошують як `private`.

Оголошення полів як `private` є інструментом забезпечення інкапсуляції. Це повністю перекриває можливість прямої некоректної модифікації внутрішнього стану об’єкта ззовні (наприклад, встановлення негативного віку або

ціни). Контроль доступу та валідація даних у такому випадку примусово здійснюються через відкриті публічні методи інтерфейсу.

25. Що таке наслідування.

Наслідування — це принцип ООП, який дозволяє створити новий (дочірній) клас на основі вже існуючого (базового) класу. Дочірній клас при цьому автоматично переймає структуру полів та методи батьківського класу, отримуючи можливість розширювати їх або перевизначати їхню поведінку, що забезпечує повторне використання коду.

26. Для чого використовується ключове слово `extends`.

Ключове слово `extends` використовується в оголошенні класу для явного зазначення того, що цей клас є дочірнім (підкласом) і успадковує функціональність іншого конкретного суперкласу (наприклад, `public class Student extends Person`).

27. Що таке поліморфізм.

Поліморфізм — це принцип ООП, який дозволяє використовувати одне й те саме посилання базового типу для взаємодії з об’єктами різних дочірніх класів. Завдяки механізму пізнього зв’язування, віртуальна машина Java (JVM) під час виконання програми автоматично визначає фактичний тип об’єкта і викликає його власну перевизначену версію методу.

28. Яку роль відіграє конструктор дочірнього класу та ключове слово `super`.

Конструктор дочірнього класу відповідає за повну ініціалізацію об’єкта. Ключове слово `super` використовується для виклику конструктора батьківського класу (наприклад, `super(name)`) і обов’язково повинно стояти першим рядком. Це гарантує, що спочатку в пам’яті будуть коректно створені та ініціалізовані базові поля суперкласу, і лише потім виконається специфічна логіка дочірнього компонента.

7 ІНФОРМАЦІЙНІ РЕСУРСИ ТА ПОСИЛАННЯ НА ВИХІДНИЙ КОД

Згідно з вимогами до здачі лабораторної роботи, повний проект вихідного коду розробленого додатку розміщено у віддаленому репозиторії системи контролю версій.

– **Посилання на Git-репозиторій проєкту:**
`https://github.com/nasuehfw/oop-lab3.git`

– **Електронний каталог спільного доступу:**
https://drive.google.com/drive/folders/1mIulRQSfoHAyQ2GnqdQbwGz2dFX-5VGK?usp=drive_link